

1337 / 42 Network — Piscine Roadmap

Shell00 → C13, Rush00-02 & BSQ — what each stage teaches, and where backtracking shows up

Overview

1337 (Morocco) runs the same core curriculum as the 42 Network. Before the main cursus, every student goes through the **Piscine** ("pool"): a ~4-week bootcamp of daily solo C exercises (C00-C13) plus two shell exercises (Shell00-01), three team weekend projects (Rush00-02), and a final team project (BSQ). No teachers, no lectures — subjects (PDF) are handed out, you write code, peers + an automated checker (Moulinette) grade it against the Norm (strict style rules).

Shell00 & Shell01 — Unix / Shell basics

Shell00: navigating the filesystem, basic commands (ls, cd, cp, mv, rm, grep, find), permissions, redirections (>, >>, |), the shell environment.

Shell01: more advanced scripting — pipes, process control, regexes, writing small bash scripts. Not always mandatory to pass, but it unlocks good habits (and the terminal fluency) you need for everything after.

C00 → C13 — the daily C track

Day	Theme	What it actually teaches
C00	Intro to C	Compiling (gcc), write/read, your first functions, exit codes. Getting comfortable with the edit-compile-run loop.
C01	Pointers	Addresses, dereferencing, pointer arithmetic, passing by reference. The concept most beginners get stuck on.
C02	Strings & Arrays	Manipulating char arrays without string.h, indexing, basic array logic.
C03	String Manipulation	Reimplementing strlen/strcpy/strcmp-style functions, building your own tiny libc pieces.
C04	Integers & Bases	Number bases (binary/octal/hex), bitwise-adjacent logic, converting numbers to strings and back.
C05	Iteration & Recursion	For/while loops vs recursive functions; recursive math (factorial, power, Fibonacci-style exercises). This is the conceptual seed for backtracking later.
C06	Argc & Argv	Reading command-line arguments, parsing input, writing small CLI tools.
C07	Memory Allocation	malloc/free, dynamic arrays, the classic memory leaks and segfaults you learn to debug here.
C08	Headers, Macros & Structs	Splitting code into .h/.c files, #define macros, struct design.
C09	Libraries & Makefile	Turning your C00-C08 functions into a static library (libft-style) with a proper Makefile.
C10	Recreating Programs	Rebuilding real Unix utilities/behaviours from scratch — bigger, multi-file programs.
C11	Function Pointers	Pointers to functions, callback-style design, tables of function pointers (dispatchers).
C12	Linked Lists	Building and manipulating singly/doubly linked lists — insert, delete, traverse, sort.
C13	Binary Search Trees	BST construction, insertion, search, traversal (in/pre/post-order) — the final data-structure day.

Rush00, Rush01, Rush02 — team weekend projects

Rush00 (after ~week 1): 2-3 person team, print ASCII-art rectangles/patterns with a twist per group. Pure control flow — nested loops, conditionals, write().

Rush01 (after ~week 2, once C05-C09 are behind you): a constraint-solving puzzle — most campuses use a 4x4 "Skyscraper"-style grid (place values 1-4 respecting visibility clues on the edges) or a Sudoku-style variant. **This is the project where backtracking is explicitly required.**

Rush02 (final weekend): a bigger algorithmic project — commonly converting an arbitrary large number into its written-out form using a dictionary file, or an equivalent parsing/algorithm challenge.

BSQ — final project

"Biggest Square": given a 2D grid of empty/obstacle cells (read from stdin and from files), find and mark the largest square that fits. Graded on both correctness and performance (speed + memory), so brute-force backtracking over every square is usually too slow — most solutions move to a dynamic-programming approach instead. It's the point where the Piscine pushes you from "make it work" to "make it fast."

Where exactly is backtracking?

Rush01 is the backtracking project. The groundwork is laid earlier in C05 (recursion), but Rush01 is where you implement real recursive backtracking: try a value, recurse, and if a later placement fails, undo ("backtrack") and try the next option until the grid is fully and validly solved. BSQ can reuse the same try/undo mindset, but is normally solved with DP for performance reasons rather than pure backtracking.

Source

Community-compiled 1337/42 Piscine study roadmap (C, Unix Shell, Git) —

<https://es.scribd.com/document/953350588/1337-Piscine-Study-Roadmap-C-Unix-Shell-And-Git>

Cross-checked against public 1337/42 Piscine repos and subject archives on GitHub (day-by-day breakdowns, Rush/BSQ subjects) and the 1337 Coding School Wikipedia page.